

# CFT Properties – Problem Patterns and Implementation Steps

## Convention Used

Throughout this document, use the Continuous Fourier Transform (CFT) convention

$$X(f) = \int_{-\infty}^{\infty} x(t)e^{-j2\pi ft} dt$$

and the inverse transform

$$x(t) = \int_{-\infty}^{\infty} X(f)e^{j2\pi ft} df.$$

For numerical implementation, the CFT is approximated by numerical integration:

$$X(f_k) \approx \text{trapz} \left( x(t)e^{-j2\pi f_k t}, t \right).$$

The generic Python CFT function is:

```
def compute_cft(t, x, f):
    t = np.asarray(t)
    x = np.asarray(x)
    f = np.asarray(f)

    exponential = np.exp(
        -1j * 2 * np.pi * f[:, None] * t[None, :]
    )

    integrand = exponential * x[None, :]

    X = np.trapezoid(
        integrand,
        t,
        axis=1
    )

    return X
```

The key idea is that the frequency axis is kept on axis 0 and the time axis on axis 1, so the integration is performed along `axis=1`.

## 1 Master List of CFT Properties

The following properties are the most useful ones for numerical programming problems.

| Property          | Time-domain operation            | Frequency-domain result                 |
|-------------------|----------------------------------|-----------------------------------------|
| Linearity         | $ax_1(t) + bx_2(t)$              | $aX_1(f) + bX_2(f)$                     |
| Time shift        | $x(t - t_0)$                     | $e^{-j2\pi f t_0} X(f)$                 |
| Frequency shift   | $x(t)e^{j2\pi f_0 t}$            | $X(f - f_0)$                            |
| Cosine modulation | $x(t) \cos(2\pi f_0 t)$          | $\frac{1}{2}[X(f - f_0) + X(f + f_0)]$  |
| Sine modulation   | $x(t) \sin(2\pi f_0 t)$          | $\frac{1}{2j}[X(f - f_0) - X(f + f_0)]$ |
| Time reversal     | $x(-t)$                          | $X(-f)$                                 |
| Time scaling      | $x(at)$                          | $\frac{1}{ a } X(f/a)$                  |
| Differentiation   | $\frac{dx(t)}{dt}$               | $j2\pi f X(f)$                          |
| Second derivative | $\frac{d^2 x(t)}{dt^2}$          | $-(2\pi f)^2 X(f)$                      |
| Integration       | $\int_{-\infty}^t x(\tau) d\tau$ | $\frac{X(f)}{j2\pi f}$ (with DC caveat) |
| Multiplication    | $x_1(t)x_2(t)$                   | $X_1(f) * X_2(f)$                       |
| Convolution       | $x_1(t) * x_2(t)$                | $X_1(f)X_2(f)$                          |
| Parseval          | $\int  x(t) ^2 dt$               | $\int  X(f) ^2 df$                      |
| Conjugation       | $x^*(t)$                         | $X^*(-f)$                               |

## 2 1. Linearity

### Property

If

$$y(t) = ax_1(t) + bx_2(t),$$

then

$$Y(f) = aX_1(f) + bX_2(f).$$

### How to implement

1. Create the common time array  $t$ .
2. Create  $x_1(t)$  and  $x_2(t)$ .
3. Create

$$y = ax_1 + bx_2.$$

4. Compute  $Y(f)$  directly using `compute_cft`.
5. Separately compute  $X_1(f)$  and  $X_2(f)$ .
6. Construct

$$Y_{\text{theory}} = aX_1 + bX_2.$$

7. Compare  $Y$  and  $Y_{\text{theory}}$  using magnitude and phase MSE.

```
x1 = ...
x2 = ...

y = a * x1 + b * x2

Y = compute_cft(t, y, f)
```

```
X1 = compute_cft(t, x1, f)
X2 = compute_cft(t, x2, f)

Y_theory = a * X1 + b * X2
```

## 3 2. Time Shift

### Property

If

$$y(t) = x(t - t_0),$$

then

$$Y(f) = X(f)e^{-j2\pi ft_0}.$$

### How to implement

1. Define  $x(t)$ .
2. Define the shifted signal

$$y(t) = x(t - t_0).$$

3. Compute  $X(f)$ .
4. Compute the CFT of the actual shifted signal.
5. Construct the theoretical result

$$Y_{\text{theory}} = X(f)e^{-j2\pi ft_0}.$$

6. Compare the two spectra.

```
x = ...
t0 = 1.0

y = x_function(t - t0)

X = compute_cft(t, x, f)
Y = compute_cft(t, y, f)

Y_theory = X * np.exp(-1j * 2 * np.pi * f * t0)
```

### Important implementation issue

If a signal is represented only by samples, it is usually better to define the shifted signal through its analytical function, or use interpolation. Do not simply use `np.roll` and call it a true continuous-time shift without accounting for the time axis.

## 4 3. Frequency Shift by Complex Exponential

### Property

If

$$y(t) = x(t)e^{j2\pi f_0 t},$$

then

$$Y(f) = X(f - f_0).$$

### How to implement

1. Create  $x(t)$ .
2. Multiply it by  $e^{j2\pi f_0 t}$ .
3. Compute the actual CFT.
4. Compute  $X(f - f_0)$  independently.
5. Compare.

```
f0 = 3

y = x * np.exp(1j * 2 * np.pi * f0 * t)

Y = compute_cft(t, y, f)

X_shifted = compute_cft(t, x, f - f0)

Y_theory = X_shifted
```

## 5 4. Cosine Modulation

### Property

$$y(t) = x(t) \cos(2\pi f_0 t)$$

gives

$$Y(f) = \frac{1}{2} [X(f - f_0) + X(f + f_0)].$$

### How to implement

1. Create  $x(t)$ .
2. Multiply by the cosine.
3. Compute  $Y(f)$  directly.
4. Compute  $X(f - f_0)$ .
5. Compute  $X(f + f_0)$ .

6. Add them and multiply by 1/2.

```
y = x * np.cos(2 * np.pi * f0 * t)

Y = compute_cft(t, y, f)

X_minus = compute_cft(t, x, f - f0)
X_plus = compute_cft(t, x, f + f0)

Y_theory = 0.5 * (X_minus + X_plus)
```

This is exactly the type of problem in which the spectrum splits into two shifted copies.

## 6 5. Sine Modulation

**Property**

$$y(t) = x(t) \sin(2\pi f_0 t)$$

gives

$$Y(f) = \frac{1}{2j} [X(f - f_0) - X(f + f_0)].$$

**Implementation**

```
y = x * np.sin(2 * np.pi * f0 * t)

Y = compute_cft(t, y, f)

X_minus = compute_cft(t, x, f - f0)
X_plus = compute_cft(t, x, f + f0)

Y_theory = (X_minus - X_plus) / (2j)
```

## 7 6. Time Reversal

**Property**

If

$$y(t) = x(-t),$$

then

$$Y(f) = X(-f).$$

**Implementation**

1. Define  $x(t)$ .
2. Evaluate the signal at  $-t$ .
3. Compute its CFT.

4. Independently compute  $X(-f)$ .

5. Compare.

```
y = x_function(-t)

Y = compute_cft(t, y, f)

Y_theory = compute_cft(t, x, -f)
```

## 8 7. Time Scaling

### Property

For

$$y(t) = x(at),$$

the CFT is

$$Y(f) = \frac{1}{|a|} X\left(\frac{f}{a}\right).$$

### Implementation

```
a = 2

y = x_function(a * t)

Y = compute_cft(t, y, f)

Y_theory = (
    1 / abs(a)
) * compute_cft(t, x, f / a)
```

### Interpretation

If  $|a| > 1$ , the signal becomes narrower in time and its spectrum becomes wider.

If  $0 < |a| < 1$ , the signal becomes wider in time and its spectrum becomes narrower.

## 9 8. Differentiation

### Property

If

$$y(t) = \frac{dx(t)}{dt},$$

then

$$Y(f) = j2\pi f X(f).$$

## Implementation

There are two possible numerical approaches.

### Approach A: analytical derivative

If the original signal is known analytically, define its derivative analytically.

```
x = np.exp(-t**2)

dx = -2 * t * np.exp(-t**2)

X = compute_cft(t, x, f)
Y = compute_cft(t, dx, f)

Y_theory = 1j * 2 * np.pi * f * X
```

### Approach B: numerical derivative

```
dx = np.gradient(x, t)

X = compute_cft(t, x, f)
Y = compute_cft(t, dx, f)

Y_theory = 1j * 2 * np.pi * f * X
```

The analytical derivative generally gives a cleaner verification.

## 10 9. Second Differentiation

### Property

$$\frac{d^2 x(t)}{dt^2} \leftrightarrow (j2\pi f)^2 X(f)$$

so

$$\mathcal{F} \left\{ \frac{d^2 x}{dt^2} \right\} = -(2\pi f)^2 X(f).$$

## Implementation

```
d2x = ... # analytical second derivative

X = compute_cft(t, x, f)
Y = compute_cft(t, d2x, f)

Y_theory = -(2 * np.pi * f)**2 * X
```

## 11 10. Integration

### Property

If

$$y(t) = \int_{-\infty}^t x(\tau) d\tau,$$

then, under the usual conditions,

$$Y(f) = \frac{X(f)}{j2\pi f}$$

with special care required at  $f = 0$  and for signals with nonzero DC component.

## Implementation

For a finite sampled signal, first construct the cumulative integral.

```
from scipy.integrate import cumulative_trapezoid

y = cumulative_trapezoid(
    x,
    t,
    initial=0
)

X = compute_cft(t, x, f)
Y = compute_cft(t, y, f)

Y_theory = X / (1j * 2 * np.pi * f)
```

Do not blindly divide by zero at  $f = 0$ . Mask or handle the DC point separately.

## 12 11. Convolution Theorem

### Property

If

$$y(t) = (x_1 * x_2)(t)$$

where

$$y(t) = \int_{-\infty}^{\infty} x_1(\tau) x_2(t - \tau) d\tau,$$

then

$$Y(f) = X_1(f) X_2(f).$$

## Implementation

This is the other major problem pattern from the assignment.

1. Create sampled  $x_1(t)$  and  $x_2(t)$ .
2. Numerically calculate their convolution.
3. Compute the CFT of the numerical convolution.
4. Independently compute  $X_1(f)$ .

5. Independently compute  $X_2(f)$ .
6. Multiply them element-by-element.
7. Compare.

```
def conv_numeric(t, x1, x2):
    y = np.zeros(len(t))

    for i in range(len(t)):
        shifted_time = t[i] - t

        x2_shifted = np.interp(
            shifted_time,
            t,
            x2,
            left=0,
            right=0
        )

        integrand = x1 * x2_shifted

        y[i] = np.trapezoid(
            integrand,
            t
        )

    return y

y = conv_numeric(t, x1, x2)

Y = compute_cft(t, y, f)

X1 = compute_cft(t, x1, f)
X2 = compute_cft(t, x2, f)

Y_theory = X1 * X2
```

Note that  $X1 * X2$  is element-wise multiplication because each frequency  $f_k$  must be multiplied by the corresponding value at the same frequency:

$$Y(f_k) = X_1(f_k)X_2(f_k).$$

## 13 12. Multiplication in Time

### Property

If

$$y(t) = x_1(t)x_2(t),$$

then

$$Y(f) = X_1(f) * X_2(f)$$

where the convolution in frequency is

$$Y(f) = \int_{-\infty}^{\infty} X_1(\nu)X_2(f - \nu)d\nu.$$

## Implementation

This is the dual of the convolution theorem.

1. Multiply  $x_1(t)$  and  $x_2(t)$  in time.
2. Compute the CFT of the product.
3. Compute  $X_1(f)$  and  $X_2(f)$ .
4. Numerically convolve the two spectra in frequency.
5. Remember that the frequency-domain convolution integral is over frequency, not time.
6. Compare the two results.

```
y = x1 * x2

Y = compute_cft(t, y, f)

X1 = compute_cft(t, x1, f)
X2 = compute_cft(t, x2, f)

# Frequency-domain convolution must be
# implemented as a numerical integral:
#
# Y_theory(f) =
# integral X1(nu) X2(f-nu) dnu
#
# using a suitable frequency grid.
```

**Important:** With the convention used in this document, there is no additional scale factor in the multiplication/convolution pair.

## 14 13. Conjugation

### Property

$$x^*(t) \leftrightarrow X^*(-f).$$

Therefore,

$$\mathcal{F}\{x^*(t)\} = X^*(-f).$$

### Implementation

```
y = np.conj(x)

Y = compute_cft(t, y, f)

Y_theory = np.conj(
    compute_cft(t, x, -f)
)
```

For a real-valued signal, this leads to the important Hermitian symmetry:

$$\boxed{X(-f) = X^*(f)}.$$

## 15 14. Parseval's Theorem

### Property

For the CFT convention used here,

$$\boxed{\int_{-\infty}^{\infty} |x(t)|^2 dt = \int_{-\infty}^{\infty} |X(f)|^2 df}.$$

### Implementation

```
X = compute_cft(t, x, f)

energy_time = np.trapezoid(
    np.abs(x)**2,
    t
)

energy_frequency = np.trapezoid(
    np.abs(X)**2,
    f
)

energy_error = abs(
    energy_time - energy_frequency
)
```

The two values should become closer as the time and frequency ranges and sampling densities become sufficiently large.

## 16 15. Duality

### Property

With the convention

$$x(t) \leftrightarrow X(f),$$

duality gives

$$\boxed{X(t) \leftrightarrow x(-f)}.$$

This property is especially useful when a known transform pair is given and the problem asks you to generate a second transform pair.

### Implementation idea

If a known pair is

$$x(t) \leftrightarrow X(f),$$

then construct a new time-domain signal whose values are the original spectrum evaluated at the time variable. Then compare its CFT with the original signal evaluated at  $-f$ .

```
# Conceptual implementation:
new_signal = X_function(t)

Y = compute_cft(t, new_signal, f)

Y_theory = x_function(-f)
```

Care is needed because the numerical grids for the original time and frequency variables may not have identical ranges and resolutions.

## 17 16. Real Signal and Hermitian Symmetry

If  $x(t)$  is real-valued, then

$$X(-f) = X^*(f).$$

Therefore,

$$|X(-f)| = |X(f)|$$

and

$$\angle X(-f) = -\angle X(f)$$

apart from phase wrapping.

### Implementation

```
X_positive = compute_cft(t, x, f)
X_negative = compute_cft(t, x, -f)

symmetry_error = np.max(
    np.abs(
        X_negative - np.conj(X_positive)
    )
)
```

This is often a useful verification problem because it checks whether the numerical CFT has been implemented correctly.

## 18 17. Even and Odd Signals

### Even signal

If

$$x(t) = x(-t)$$

and  $x(t)$  is real, then

$$X(f) \text{ is real and even.}$$

Thus,

$$\text{Im}\{X(f)\} \approx 0.$$

## Odd signal

If

$$x(t) = -x(-t)$$

and  $x(t)$  is real, then

$X(f)$  is imaginary and odd.

Thus,

$$\operatorname{Re}\{X(f)\} \approx 0.$$

## Implementation

```
X = compute_cft(t, x, f)

real_error = np.max(np.abs(np.real(X)))
imag_error = np.max(np.abs(np.imag(X)))
```

The expected small values depend on numerical resolution and truncation.

## 19 18. Amplitude and Phase Effects of Time Shift

A time shift

$$x(t - t_0)$$

produces

$$Y(f) = X(f)e^{-j2\pi ft_0}.$$

Therefore,

$$|Y(f)| = |X(f)|$$

while the phase changes by

$$-2\pi ft_0.$$

This makes a good programming problem:

1. Calculate the CFT of the original signal.
2. Calculate the CFT of the shifted signal.
3. Compare magnitudes.
4. Compare phases.
5. Verify that magnitude MSE is very small while the phase follows the predicted linear shift.

## 20 19. Frequency Shift and Magnitude Spectrum

For

$$y(t) = x(t)e^{j2\pi f_0 t},$$

we have

$$Y(f) = X(f - f_0).$$

Therefore the magnitude spectrum simply moves:

$$|Y(f)| = |X(f - f_0)|.$$

This can be implemented by comparing the direct CFT with a shifted frequency evaluation.

## 21 20. Cosine Modulation as Two Frequency Copies

For

$$y(t) = x(t) \cos(2\pi f_0 t),$$

the spectrum becomes

$$Y(f) = \frac{1}{2}X(f - f_0) + \frac{1}{2}X(f + f_0).$$

A useful problem can ask you to verify all three:

1. location of the positive-frequency copy,
2. location of the negative-frequency copy,
3. factor 1/2 in their amplitudes.

The implementation is:

```
X_minus = compute_cft(t, x, f - f0)
X_plus = compute_cft(t, x, f + f0)

Y_theory = 0.5 * (X_minus + X_plus)
```

## 22 21. A Standard MSE Verification Function

For most property-verification problems, the following function is useful.

```
def calculate_mse(array1, array2):

    mse_mag = np.mean(
        (np.abs(array1) - np.abs(array2))**2
    )

    phase_diff = (
        np.angle(array1)
        - np.angle(array2)
        + np.pi
    ) % (2 * np.pi) - np.pi
```

```

mse_phase = np.mean(
    phase_diff**2
)

return mse_mag, mse_phase

```

For phase, it is often better to ignore points where the magnitude is almost zero:

```

threshold = 1e-6 * np.max(np.abs(reference))

mask = np.abs(reference) > threshold

phase_diff = (
    np.angle(actual[mask])
    - np.angle(reference[mask])
    + np.pi
) % (2*np.pi) - np.pi

masked_phase_mse = np.mean(
    phase_diff**2
)

```

## 23 22. General Problem-Solving Template

Almost every CFT-property programming question can be solved using the same structure.

### Step 1: Define the time grid

```
t = np.linspace(t_start, t_end, N)
```

### Step 2: Define the frequency grid

```
f = np.linspace(f_start, f_end, M)
```

### Step 3: Define the original signals

```

x1 = ...
x2 = ...

```

### Step 4: Perform the time-domain operation

For example:

```
y = 2*x1 + x2
```

or

```
y = x1 * np.cos(2*np.pi*f0*t)
```

or use a numerical convolution function.

### Step 5: Compute the actual CFT

```
Y = compute_cft(t, y, f)
```

## Step 6: Compute the theoretical spectrum independently

Use the relevant CFT property.

```
Y_theory = ...
```

## Step 7: Compare

```
mse_mag, mse_phase = calculate_mse(  
    Y,  
    Y_theory  
)
```

## Step 8: Plot

At minimum, compare

- magnitude of the actual and theoretical spectra,
- phase of the actual and theoretical spectra.

## 24 23. Which Properties Are Most Likely to Become Coding Problems?

For a practical CFT programming assignment, prioritize these first:

1. **Linearity**
2. **Time shifting**
3. **Time reversal**
4. **Frequency shifting**
5. **Cosine modulation**
6. **Sine modulation**
7. **Time scaling**
8. **Convolution theorem**
9. **Multiplication/convolution duality**
10. **Differentiation**
11. **Parseval's theorem**
12. **Conjugation and Hermitian symmetry**

The first eight are especially suitable for exactly the style of problems in which you are asked to construct a signal, compute its numerical CFT, construct the predicted spectrum from a CFT property, and verify the result using MSE.

## Final Mental Model

For every property-verification problem, think:

Create  $y(t) \longrightarrow$  compute actual  $Y(f)$

and separately

Use the CFT property  $\longrightarrow$  construct theoretical  $Y(f)$

Then:

Actual spectrum    vs    Theoretical spectrum

using magnitude MSE, phase MSE, and plots.

The most important implementation distinction is:

$*$  in NumPy = element-wise multiplication

whereas

@ in NumPy = matrix multiplication.

For CFT problems, both can appear, but they represent different mathematical operations.